Code : LoRa bridge (server.js)

Project reference: STN22_Thoracic belt for respiratory rate measurement
Authors: Hakkou Dounia
Date: 2025–2026 © SIS TEAM NANCY

File header

```
/*
  Description: Node.js bridge between the Heltec LoRa receiver (USB serial port)
  and the RespiraMed REST API hosted on Hostinger.
  Reads RPM values emitted by the Arduino firmware over UART, then POSTs each
  measurement to /measurements only when an active session is running.
  Session lifecycle (start/stop) is controlled via stdin commands typed in the terminal.

  Runtime : Node.js 18+ (ESM modules, 'type':'module' in package.json)
  Dependencies : serialport, @serialport/parser-readline, axios, dotenv

  Project reference: STN22_Thoracic belt FR
  Authors: Hakkou Dounia
  Date: 2025-2026
  © SIS TEAM NANCY
*/
```

Imports

```
// SerialPort: opens and reads the USB serial port connected to the Heltec LoRa receiver
import { SerialPort } from 'serialport';
// ReadlineParser: buffers raw bytes from the serial port and emits complete lines
// delimited by \r\n (carriage return + line feed), matching the Arduino firmware output
import { ReadlineParser } from '@serialport/parser-readline';
// axios: HTTP client used to POST measurements and session commands to the REST API
import axios from 'axios';
// dotenv: loads environment variables from a .env file at project root
// Allows COM_PORT and API_URL to be configured per machine without editing source code
import dotenv from 'dotenv';

dotenv.config();  // Must be called before reading process.env
```

Configuration

```
// Target REST API base URL.
// Set API_URL in .env to override the production URL during local development.
const API_URL = process.env.API_URL || 'https://sisteamnancy.fr/api-respiramed';

// Serial port path of the Heltec WiFi LoRa 32 V3 receiver connected via USB.
// On Windows the port appears as COM5 (or COM3, COM6... check Device Manager).
// On Linux/Raspberry Pi, uncomment the ttyUSB0 line and comment out the COM5 line.
const COM_PORT = process.env.COM_PORT || 'COM5'; // Windows
// const COM_PORT = process.env.COM_PORT || '/dev/ttyUSB0'; // Linux/Raspberry Pi

// Baud rate must match the value configured in the Arduino firmware.
// The Heltec firmware uses 115200 bps for USB serial communication.
const BAUD_RATE = 115200;
```

Session state variables

```
// Tracks the currently active measurement session.
// Set to the sessionId returned by POST /set-patient when a session is started.
// Reset to null when the session is stopped or the process exits.
let currentSessionId = null;

// Tracks the UUID of the patient associated with the active session.
// Required as a payload field for POST /measurements.
let currentPatientId = null;
```

Serial port initialisation

```
// Create a SerialPort instance for the configured COM port.
// autoOpen: false means the port is NOT opened immediately on construction.
// It will be opened explicitly via port.open() below so we can handle the error.
const port = new SerialPort({
  path: COM_PORT,
  baudRate: BAUD_RATE,
  autoOpen: false   // Defer opening so we can catch errors gracefully
});

// Pipe the raw serial stream through ReadlineParser.
// The parser buffers incoming bytes and emits one string per complete line (\r\n).
// This matches the format produced by the Heltec Arduino firmware.
const parser = port.pipe(new ReadlineParser({ delimiter: '\r\n' }));

// Explicitly open the serial port.
port.open((err) => {
  if (err) {
    // Opening failed: port may be busy, not connected, or path is wrong.
    // Print guidance and exit so the user knows to check the port.
    console.error('Erreur ouverture port série:', err.message);
    console.log('Ports disponibles : Exécute "npx @serialport/list" pour voir la liste');
    process.exit(1);  // Non-zero exit code signals failure to the shell
  }
  console.log(`Port série ouvert sur ${COM_PORT} @ ${BAUD_RATE} baud`);
});
```

Data handler: parser.on('data')

```
// Called every time the ReadlineParser emits a complete line from the serial port.
// The Arduino firmware sends lines in one of two formats:
// - 'DATA:RPM:XX.X'(direct serial debug output)
// - 'LoRa TX: R:XX.X' (LoRa TX echo format used in the receiver firmware)
// The regex captures the numeric RPM value from either format.
parser.on('data', async (line) => {
  console.log('Reçu:', line);  // Log every received line for debugging

  // Try to extract the RPM value using a case-insensitive regex.
  // Capture group 1 contains the decimal number (e.g. '18.5').
  const match = line.match(/(?:DATA:RPM:|LoRa\s+TX:\s+R:)([\d.]+)/i);

  if (match) {
    const rpm = parseFloat(match[1]);  // Convert the captured string to a float
    console.log(`RPM détecté: ${rpm}`);

    // Only record the measurement if a session is currently active.
    // If no session is running, the LoRa data is silently discarded.
    if (currentSessionId && currentPatientId) {
      try {
        // POST the measurement to the backend with all required fields.
        await axios.post(`${API_URL}/measurements`, {
          patientId: currentPatientId,  // Links the measurement to the correct patient
          sessionId: currentSessionId,  // Links the measurement to the active session
          rpm: rpm                      // The measured respiratory rate in rpm
        });
        console.log(`Mesure enregistrée: ${rpm} RPM`);
      } catch (err) {
        // Log the server error message if available, otherwise the network error.
        console.error('Erreur envoi mesure:', err.response?.data || err.message);
      }
    } else {
      // No active session: discard the measurement and log a warning.
      console.log('Pas de session active - mesure ignorée');
    }
  }
  // If regex does not match, the line is not a measurement (e.g. debug output),ignore.
});
```

Error handler: port.on('error')

```
// Handles low-level serial port errors that occur after the port is open
// (e.g. device disconnected, USB cable pulled, driver crash).
// The error is logged but the process is not killed, allowing for reconnection.
port.on('error', (err) => {
  console.error('Erreur port série:', err.message);
});
```

Stdin command handler

```
// Read commands typed by the operator in the terminal.
// This allows the operator to start/stop sessions, check status, and exit.
// setEncoding ensures the input is decoded as UTF-8 text, not a Buffer.
process.stdin.setEncoding('utf8');
process.stdin.on('data', async (input) => {
  const cmd = input.trim().toLowerCase();  // Normalise whitespace and case

  if (cmd.startsWith('start ')) {
    // Extract the patient UUID from 'start <patientId>'
    const patientId = cmd.split(' ')[1];
    try {
      // POST /set-patient tells the backend which patient is active.
      // The response contains the newly created sessionId.
      const res = await axios.post(`${API_URL}/set-patient`, { patientId });
      currentSessionId = res.data.sessionId;  // Store for use in data handler
      currentPatientId = patientId;
      console.log(`Session démarrée: ${currentSessionId} pour patient ${patientId}`);
    } catch (err) {
      console.error('Erreur démarrage session:', err.response?.data || err.message);
    }

  } else if (cmd === 'stop') {
    try {
      // POST /stop-session closes the session on the backend.
      // No body is required — the backend closes whatever session is active.
      await axios.post(`${API_URL}/stop-session`);
      console.log(`Session ${currentSessionId} arrêtée`);
      // Reset local state so new measurements are ignored until next start
      currentSessionId = null;
      currentPatientId = null;
    } catch (err) {
      console.error('Erreur arrêt session:', err.response?.data || err.message);
    }

  } else if (cmd === 'status') {
    // Print the current session and patient IDs to the terminal.
    console.log(`Session active: ${currentSessionId || 'Aucune'}`);
    console.log(`Patient ID: ${currentPatientId || 'Aucun'}`);

  } else if (cmd === 'help') {
    // Print a summary of available commands.
    console.log(`
Commandes disponibles:
  start <patientId>  - Démarre une session pour un patient
  stop               - Arrête la session active
  status             - Affiche l'état actuel
  help               - Affiche cette aide
  exit               - Quitte le programme
    `);

  } else if (cmd === 'exit') {
    console.log(' Arrêt du lora...');
    port.close();    // Close the serial port cleanly before exiting
    process.exit(0); // Exit with code 0 (normal termination)
  }
});
```

Startup banner

```
// Print a startup summary so the operator knows the server is running
// and which COM port and API it is connected to.
console.log(`
LoRa Listener démarré
```

```
Port: ${COM_PORT}
API: ${API_URL}

`);
```